



# Project Documentation: Event Ticketing & Reservation System

## 1. Executive Summary

The **Event Ticketing & Reservation System** is a full-stack web application designed to streamline the process of discovering events (concerts, movies, sports, conferences) and purchasing tickets online. Built using a modern **decoupled architecture**, the system utilizes **Laravel (REST API)** on the backend for robust data management and business logic, and **React.js** on the frontend to deliver a seamless, dynamic, and responsive Single Page Application (SPA) user experience.

### Key Features

- **Interactive Seat Selection:** A dynamic frontend grid mapping the venue's layout, allowing users to view real-time seat availability and select preferred slots.
- **Concurrency Control (Seat Locking Logic):** A mechanism that prevents race conditions (two users buying the same seat) by temporarily reserving seats for 10 minutes during checkout.
- **Comprehensive Admin Dashboard:** An administrative control panel for event organizers to manage events, customize seat tiers, set pricing, and monitor real-time sales analytics.

## 2. System Architecture

The application separates the presentation layer from the data layer to ensure scalability, ease of deployment, and clean code maintenance.

- **Frontend (React.js):** Handles client-side routing and UI rendering. It uses state management (Zustand or Redux Toolkit) to track user sessions and shopping cart data. Communication with the backend is done asynchronously via HTTP requests using **Axios**.
- **Backend (Laravel API):** Functions strictly as a stateless RESTful API. It processes incoming requests, enforces data validation, handles database interactions, and returns responses in standardized **JSON format**. Authentication is secured via token-based **Laravel Sanctum**.

## 3. Database Entity Relationship Diagram (ERD)

The database schema is designed to enforce data integrity and support complex transactional relationships required for seat allocations.

[Table: users]

- id (INT, PK)
- name (VARCHAR)
- email (VARCHAR, UNIQUE)
- password (VARCHAR)

|  
| (1 to Many)  
v

[Table: bookings]

- id (INT, PK)
- user\_id (INT, FK -> users.id)
- event\_id (INT, FK -> events.id)
- total\_price (DECIMAL)
- payment\_status (VARCHAR) ---- [values: 'pending', 'paid', 'failed']

|  
| (1 to Many)  
v

[Table: booking\_items]

- id (INT, PK)
- booking\_id (INT, FK -> bookings.id)
- seat\_id (INT, FK -> seats.id)
- unit\_price (DECIMAL)

^  
| (Many to 1)

[Table: seats]

- id (INT, PK)
- event\_id (INT, FK -> events.id)
- tier\_id (INT, FK -> seat\_tiers.id)
- seat\_number (VARCHAR)
- status (VARCHAR) ---- [values: 'available', 'pending', 'booked']
- lock\_expired\_at (TIMESTAMP, NULLABLE)

^  
| (Many to 1)

[Table: seat\_tiers]

- id (INT, PK)
- event\_id (INT, FK -> events.id)
- name (VARCHAR) ---- [e.g., 'VIP', 'Premium', 'Standard']
- price (DECIMAL)

^  
| (Many to 1)

[Table: events]

- id (INT, PK)
- title (VARCHAR)
- description (TEXT)
- location (VARCHAR)
- start\_time (TIMESTAMP)
- end\_time (TIMESTAMP)

## 4. Core Business Logic & Workflows

### A. Dynamic Seat Generation

When an admin creates a new **Event** and configures its **Seat Tiers** (e.g., VIP = \$50, Standard = \$20), a backend routine automatically populates the **seats** table with individual seat records assigned a default status of **available**.

### B. The Seat Locking Mechanism (Addressing Concurrency)

1. When a user selects a seat in the React UI, a POST request is sent to the API.
2. The server checks if the seat's status is **available**. If yes, it updates the status to **pending** and sets a timestamp: **lock\_expired\_at = NOW() + 10 minutes**.
3. If another user attempts to select the same seat during this window, the API rejects the request with a **422 Unprocessable Entity** status code.
4. A background **Laravel Task Scheduler (Cron Job)** runs periodically to revert any expired **pending** seats back to **available** if the checkout process is not completed.

### C. Booking Completion and Ticket Generation

Upon receiving a successful payment confirmation, the transaction updates **bookings.payment\_status** to **'paid'** and changes the associated **seats.status** to **'booked'**. The system then generates a unique verification **QR Code** and automatically dispatches a PDF e-ticket to the user's registered email address.